# Doble DQN y DQN en duelo

Manuel Sánchez-Montañés

1. Doble DQN
2. DQN en duelo (dueling DQN)

# Doble DQN

# Doble DQN

- Idea desarrollada por investigadores de Google Deep Mind

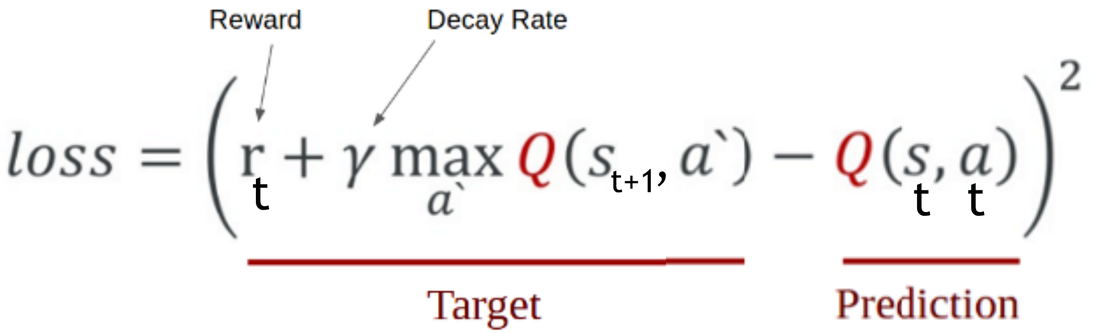
# Doble DQN

- Idea desarrollada por investigadores de Google Deep Mind

- Van Hasselt, H., Guez, A., Silver, D. (2015): “Deep Learning with Double Q-Learning”. https://arxiv.org/pdf/1509.06461.pdf

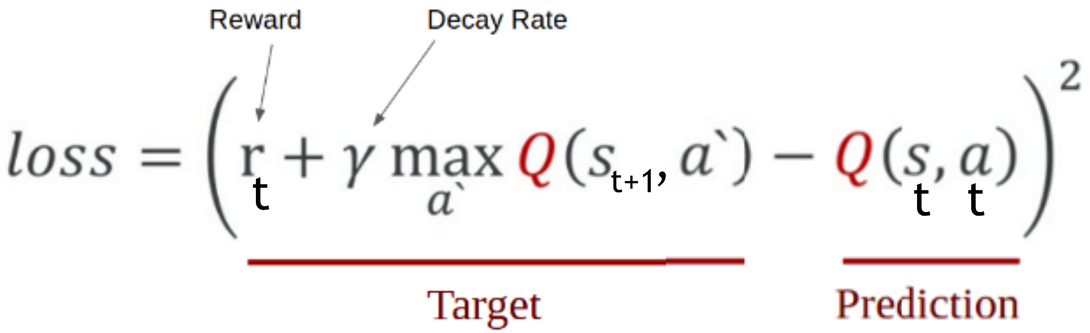
# Doble DQN

- La ecuación básica de DQN, como vimos, es:



# Doble DQN

- La ecuación básica de DQN, como vimos, es:



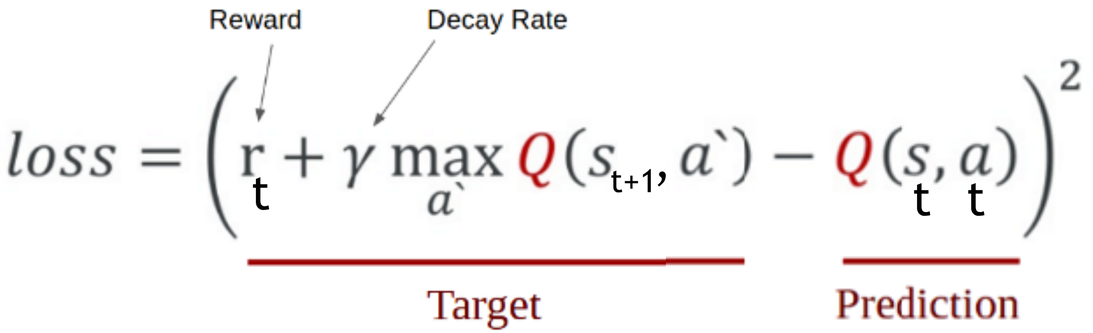

- Que se puede reescribir como:

$Y_t = r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$

$loss = (Y_t - Q(s_t, a_t))^2$

# Doble DQN

- La ecuación básica de DQN, como vimos, es:




- Que se puede reescribir como:

$Y_t = r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$

$loss = (Y_t - Q(s_t, a_t))^2$

- El operador max hace que se tienda a sobreestimar la Q

# Doble DQN

- **Solución:** se tienen dos redes **Q** y **Q’**

# Doble DQN

- **Solución:** se tienen dos redes **Q** y **Q’**
- En vez de calcular el target como:

$$Y_t = r_t + \gamma \cdot \max_{a'} \mathbf{Q}(s_{t+1}, a')$$

# Doble DQN

- **Solución:** se tienen dos redes **Q** y **Q’**
- En vez de calcular el target como:

$$Y_t = r_t + \gamma \cdot \max_{a'} \mathbf{Q}(s_{t+1}, a')$$

- Se calcula como:

$$\text{acción} = \operatorname{argmax}_{a'} \mathbf{Q}(s_{t+1}, a')$$

$$Y_t = r_t + \gamma \cdot \max_{a'} \mathbf{Q'}(s_{t+1}, \text{acción})$$

# Doble DQN

- **Solución:** se tienen dos redes **Q** y **Q’**
- En vez de calcular el target como:

$$Y_t = r_t + \gamma \cdot \max_{a'} \mathbf{Q}(s_{t+1}, a')$$

- Se calcula como:

$$\text{acción} = \operatorname{argmax}_{a'} \mathbf{Q}(s_{t+1}, a')$$

$$Y_t = r_t + \gamma \cdot \max_{a'} \mathbf{Q'}(s_{t+1}, \text{acción})$$

- La red **Q** se actualiza en cada paso

# Doble DQN

- **Solución:** se tienen dos redes **Q** y **Q’**
- En vez de calcular el target como:

$$Y_t = r_t + \gamma \cdot \max_{a'} \mathbf{Q}(s_{t+1}, a')$$

- Se calcula como:

$$\text{acción} = \operatorname{argmax}_{a'} \mathbf{Q}(s_{t+1}, a')$$

$$Y_t = r_t + \gamma \cdot \max_{a'} \mathbf{Q'}(s_{t+1}, \text{acción})$$

- La red **Q** se actualiza en cada paso
- **Q’** es una red que se actualiza más lentamente: cada k pasos, se copia **Q** en **Q’**. Entre medias, **Q’** no se actualiza

# Doble DQN

- **Como consecuencia, el aprendizaje es más estable**

# DQN en duelo

# DQN en duelo

- Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., de Freitas, N. (2016): “Dueling Network Architectures for Deep Reinforcement Learning”. Proceedings of the 33rt International Conference on Machine Learning. http://proceedings.mlr.press/v48/wangf16.pdf

# DQN en duelo

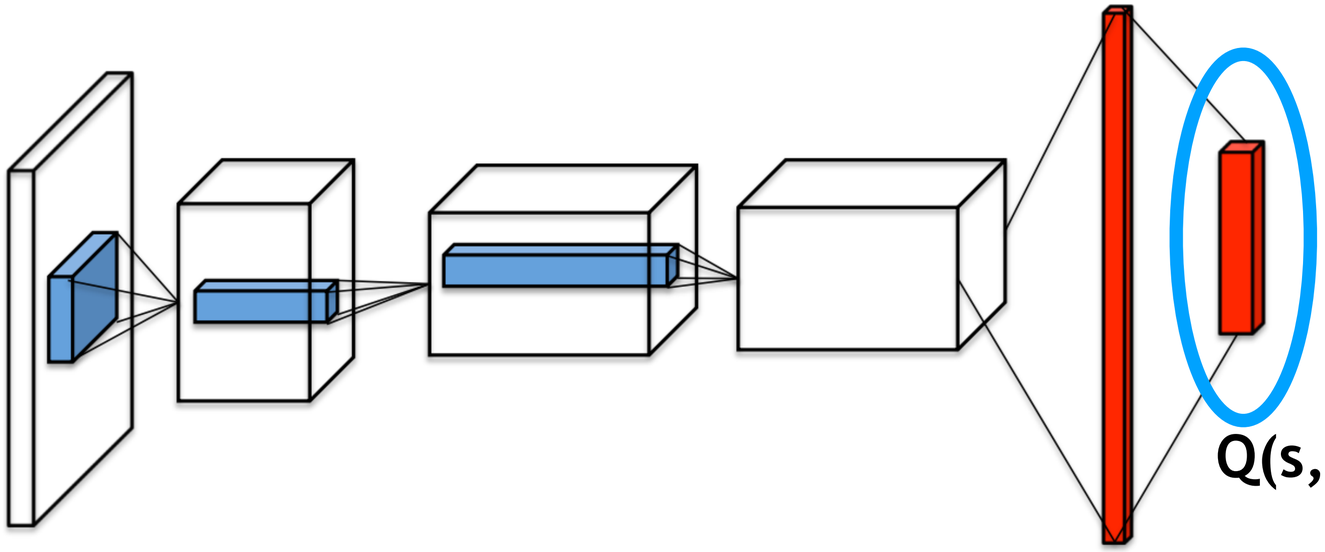
- Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., de Freitas, N. (2016): “Dueling Network Architectures for Deep Reinforcement Learning”. Proceedings of the 33rt International Conference on Machine Learning. http://proceedings.mlr.press/v48/wangf16.pdf

- La idea es que Q(s,a) se calcula como la suma de dos términos:
  - Función de valor V(s)
  - Ventaja A(s,a) de cada acción

# DQN en duelo

- Idea:

**DQN básica:**



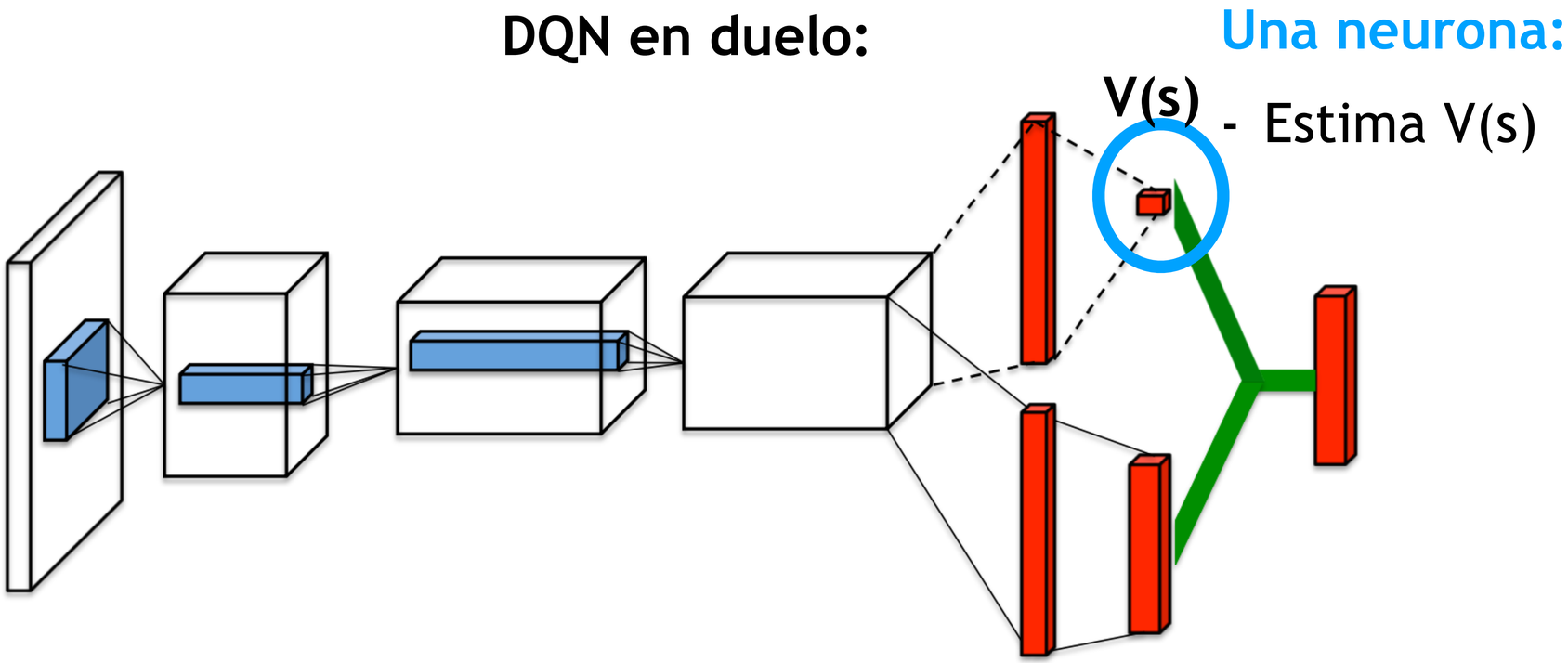

**Capa de salida:**

- Estima Q para cada acción
- Tantas neuronas de salida como acciones

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., de Freitas, N. (2016): “Dueling Network Architectures for Deep Reinforcement Learning”. Proceedings of the 33rt International Conference on Machine Learning.

# DQN en duelo

- Idea:

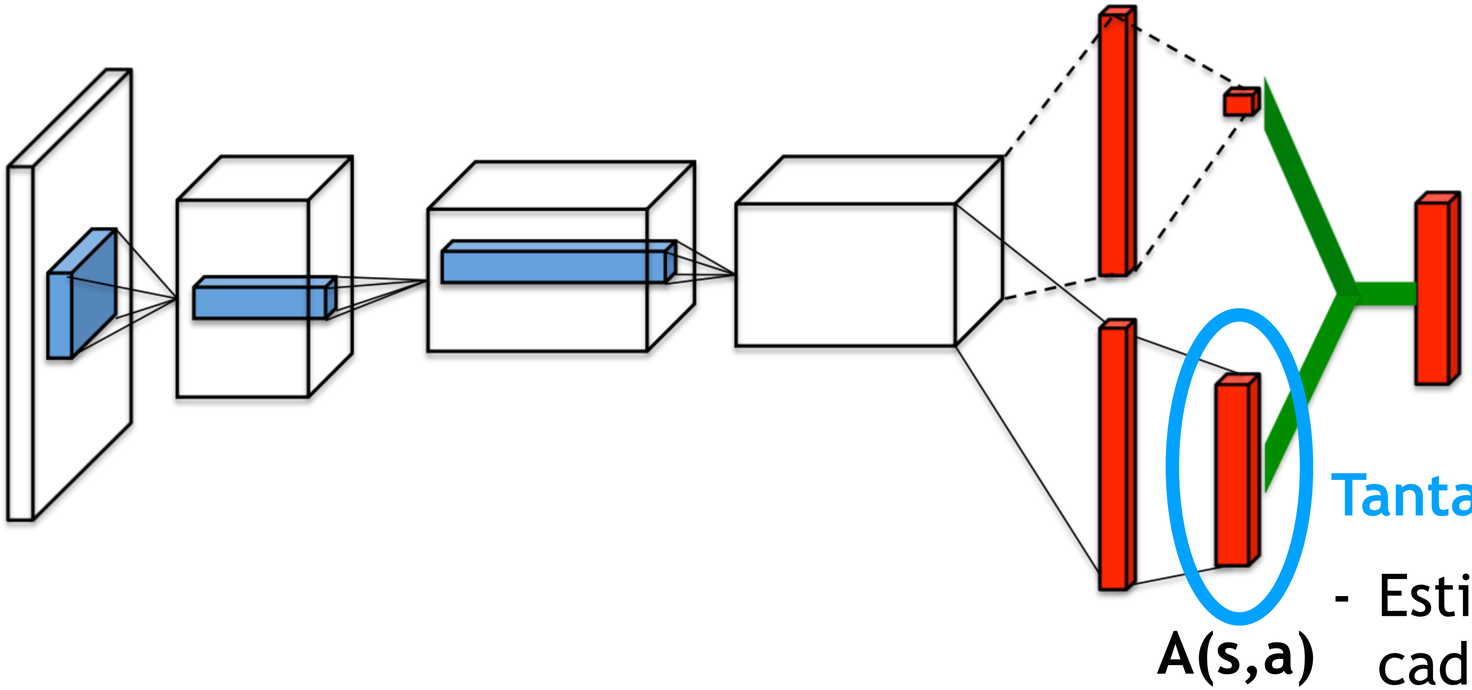



Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., de Freitas, N. (2016): “Dueling Network Architectures for Deep Reinforcement Learning”. Proceedings of the 33rt International Conference on Machine Learning.

# DQN en duelo

- Idea:

**DQN en duelo:**



**A(s,a)**

**Tantas neuronas como acciones:**

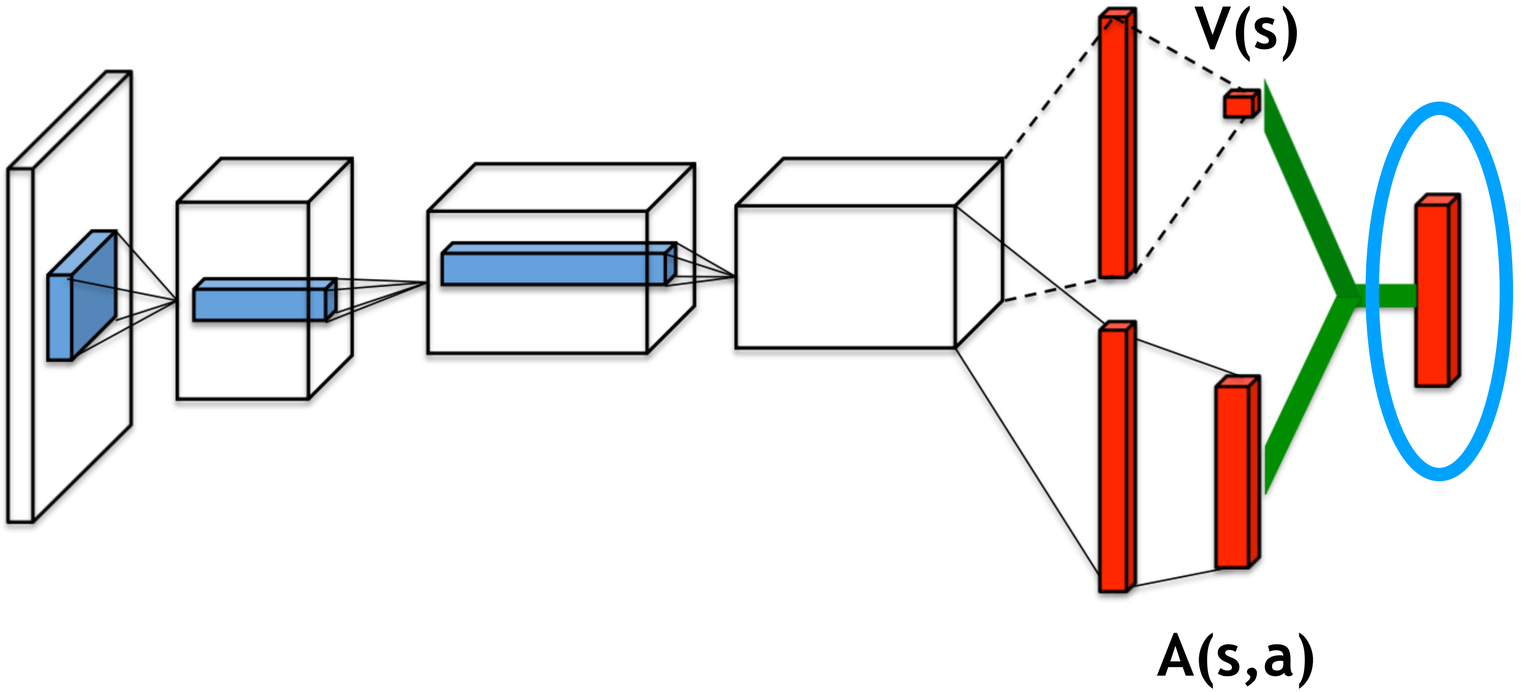
- Estiman la ventaja **A(s,a)** de cada acción sobre las otras

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., de Freitas, N. (2016): “Dueling Network Architectures for Deep Reinforcement Learning”. Proceedings of the 33rt International Conference on Machine Learning.

# DQN en duelo

- Idea:

**DQN en duelo:**




**Capa de salida:**

- Estima Q para cada acción
- Función de **V(s)** y **A(s,a)**
- Tantas neuronas de salida como acciones

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., de Freitas, N. (2016): “Dueling Network Architectures for Deep Reinforcement Learning”. Proceedings of the 33rt International Conference on Machine Learning.

# DQN en duelo

- Valoración de cada acción Q(s,a):
  - $Q(s,a) = V(s) + A(s,a) - \text{media}_{a'}(A(s,a'))$